Pattern B Operationalization: Substrate-Derived Views as Standalone Architectural Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the three legitimate composition patterns the source paper supports for adjacent AI components — Pattern B, in which an adjacent component holds a derived view of substrate content — as a standalone architectural specification, integrating the source paper's commitments to path retraceability, substrate-as-source-of-truth, the determinism contract, and tool-agnosticism into a single operational specification of what Pattern B requires.

Abstract

The CKS pattern supports three legitimate composition patterns for adjacent AI components: an adjacent component as input to a cell (Pattern A), as a derived view of substrate content (Pattern B), or as a separate concern (Pattern C). Pattern B is the legitimate position for the AI components most widely deployed in 2024–2026 systems — search indexes, vector databases, retrieval-augmented generation surfaces drawing from substrate-resident content, dashboards, materialized views, and cached projections — and it is also the position whose drift produces the most consequential failure mode the source paper preempts: an adjacent component acquiring authority it was never granted, with the substrate gradually losing source-of-truth status to a "derived" view that has become, in practice, the surface participants reach for first. A separate note in this series establishes Pattern B's existence within the three-pattern framework. This note formalizes Pattern B as a standalone architectural specification: it states the four operational components, identifies what the operationalization forces beyond the general framework, distinguishes Pattern B from Pattern A and Pattern C, names the anti-patterns that violate Pattern B specifically, and supplies an operational test with three sharpening properties for whether a deployment satisfies Pattern B as architecture.

1. Why Pattern B operationalization needs to be formalized as standalone

The CKS source paper supports three legitimate composition patterns for adjacent AI components. Pattern A positions an adjacent component as input to a cell's reasoning under an orchestration rule; Pattern B positions an adjacent component as a derived view of substrate content; Pattern C positions an adjacent component as handling a separate concern that does not affect coordination state. The three-pattern framework gives downstream implementations a vocabulary for placing adjacent components correctly; what it does not do, on its own, is specify the full operational content of any one position.

Several earlier composition-pair notes in this series have addressed specific aspects of Pattern B without consolidating them: a reproducibility-oriented composition addressing how path retraceability and the determinism contract together require a derived view to be regenerable with provenance; a vendor-independence composition addressing how tool-agnosticism and substrate-as-source-of-truth together require view-generation tooling to be portable; an adjacency-versus-substrate composition addressing how the architectural distinction between substrate and adjacent components positions any derived view on the adjacent side. Each captures a piece. None states the full architectural specification of Pattern B as one integrated commitment.

That is the work of this note. Pattern B's standalone operationalization is the integration: a single specification under which a search index, a vector database, a retrieval-augmented generation surface, a dashboard, or a materialized view qualifies as a CKS-coherent Pattern B instance, with each piece — unidirectional derivation, non-authoritative status, retraceable provenance, deterministic regenerability — stated as a component rather than as a property derived from a different pair. The strategic posture is explicit: Pattern B is the position the AI components most widely deployed in 2024–2026 systems can occupy legitimately, and the operationalization specifies what each such deployment must satisfy to remain CKS-coherent. Naming the specification as a single architectural object is what distinguishes legitimate Pattern B deployments from the drift the source paper preempts at §6.2 under the heading *context rot* — substrate content participants thought authoritative compressed, summarized, or re-embedded out of fidelity with what the substrate actually carries.

2. The four operational components

A composition is a Pattern B instance if and only if it satisfies all four of the following operational components together. Each component projects a specific commitment of the source paper onto the Pattern B position; the four together constitute the architectural specification.

(a) Unidirectional substrate-to-adjacent view derivation. The flow of view content runs in one direction only: from substrate state to the adjacent component, where it becomes the view's content. The reverse direction — view content propagating back to substrate state as authoritative — is not part of the pattern. If view changes are to affect substrate state, that effect runs through a cell under an orchestration rule, which is Pattern A applied to the view's contents as input, not a continuation of Pattern B in reverse. The unidirectionality is what makes Pattern B legible as a derivation rather than a coupling: substrate is the source, view is the result, and no route from view back to substrate exists outside cell mediation.

(b) Derived view as non-authoritative mirror. The derived view is non-authoritative for any coordination question the substrate is the source of truth for. Where substrate and view disagree on what was decided, by whom, under what authority, with what rationale, or what contradictions remain, the substrate wins by definition; the disagreement is not a question the system adjudicates but a signal that the view is stale or otherwise out of fidelity with the substrate. The source-of-truth-versus-mirror distinction the source paper carries — the substrate is the source; views are mirrors — is what this component pins down. A view treated as authoritative is not a Pattern B instance; it is the architectural failure mode this component preempts.

(c) Retraceable derivation provenance. The derivation event that produced the view's content is retraceable: a reader can determine, from substrate-side records, what substrate state was the source for a given view's content, when the derivation occurred, what derivation procedure was applied, and what

version of that procedure ran. The path-retraceability commitment the source paper imports for substrate writes (§3.1) extends to the derivation event: the view-generation step is itself an addressable substrate-side record, with the same provenance fields the accountability vocabulary requires for substrate content — generator identity, timestamp, antecedent reference, procedure reference, rationale where applicable, and version. Without retraceable provenance, the view exists but its derivation does not, and the architecture loses its ability to certify what the view was derived from at any moment.

(d) Deterministic regenerability from substrate state. The view is regenerable: given the substrate state at a point in time and the derivation procedure of record, running the procedure again yields equivalent view content modulo the categories of non-determinism the determinism contract permits — ordering of equivalent results, presentation metadata, cache encoding, and (where the derivation involves an embedding model or other LLM-side step) the natural-language wrapping or stochastic surface that does not affect the determinism of the recorded view content under its specified equivalence class. Regenerability is what makes a derived view legitimately disposable: a deployment that has lost the view, or whose view has drifted, can rebuild it from substrate state without consulting the lost view. A view whose content has accumulated state the substrate cannot regenerate has stopped being a derived view and become an independent store the deployment now has to govern as substrate, which is not the architectural position Pattern B occupies.

The four are jointly necessary and individually load-bearing. Failing any one fails Pattern B as architecture.

3. What the operationalization forces beyond the general framework

The three-pattern framework, in its general form, names Pattern B as a position; it does not, on its own, force any of (a)–(d) at the deployment level. The standalone operationalization forces each, and each force has an architectural correspondent in an already-committed CKS property. (a) is forced by the substrate–cell boundary commitment combined with AI-as-substrate-mediator: writes to substrate must run through cells under rules, so any "writeback" from a derived view becomes a cell-mediated input — Pattern A — rather than a continuation of Pattern B. (b) is forced by substrate-as-source-of-truth: only one location can answer coordination questions authoritatively, and the derived view is not that location by construction. (c) is forced by path retraceability: every architectural event affecting substrate-readable state must be reconstructable from substrate-side records, and the event of generating a view from substrate state is one such event. (d) is forced by the determinism contract: the substrate side of the governance boundary is deterministic, and a view derived deterministically from a deterministic source is itself regenerable as a contract guarantee within the contract's allowed-non-determinism categories.

The four operational components are not additional axioms; they are the projection of four already-committed CKS properties onto the Pattern B position. The standalone operationalization is the work of stating the projection as a single specification.

4. What Pattern B is NOT

The standalone treatment is precise about what Pattern B *is*. Stating what Pattern B is *not* is what prevents the misreading the operationalization preempts.

Not Pattern A. Pattern A is a cell consulting an adjacent component during the cell's execution; the cell reads from substrate, may also read from the adjacent component, and writes back to substrate under its rule. Pattern B has no cell-mediated reasoning step at the moment the view is read; the reader is reading a derived projection, not invoking a cell. The two patterns can coexist in a single deployment, but they are different positions, and conflating them loses the architectural property each provides.

Not Pattern C. Pattern C is an adjacent component handling a concern that does not affect coordination state — for example, a retrieval surface answering reference-document queries entirely outside the coordination work. Pattern B's view is *of* substrate content — coordination state is what the view derives from. The two are distinguishable by source: a Pattern B view is regenerable from substrate state because substrate state is its source; a Pattern C component has no such relation to substrate state because coordination state is not what it operates over.

Not bidirectional coupling. A view that propagates changes back to substrate without those changes running through a cell is not a Pattern B instance; it is the hidden-bidirectional-coupling anti-pattern the source paper preempts. The unidirectionality component is constitutive, not optional.

Not view-as-authority. A view treated as the source of truth is not a Pattern B instance; it is the substrate-substitute anti-pattern in the form most common to 2024–2026 deployments — a search index, a vector database, or a retrieval surface that has become the read surface participants reach for first, with substrate content gradually re-authored to match what the view returns. The non-authoritative-status component is what makes Pattern B distinguishable from this drift.

Not retrieval-as-knowledge-base. A retrieval surface used as the authoritative answer to coordination questions — what was decided, by whom, under what authority — is not a Pattern B instance regardless of how the retrieval system is technically configured. Pattern B accommodates retrieval over substrate-resident content as a legitimate position only when the retrieval surface is non-authoritative and the substrate retains source-of-truth status; the moment the retrieval surface answers coordination questions on its own authority, the position is not Pattern B and is not legitimate under any of the three patterns.

5. Anti-patterns and architectural decisions

Several anti-patterns formalized in adjacent notes instantiate as Pattern B violations specifically. The substrate-substitute anti-pattern instantiates whenever a derived view becomes the authoritative answer the system reaches for in place of substrate; it fails component (b). The agent-memory-as-source-of-truth and LLM-context-as-source-of-truth anti-patterns instantiate when an LLM-side projection of substrate content — a session memory, a retrieved-and-summarized context — is treated as authoritative; both fail (b) at the position where derived views meet model context. The external-tool-state-as-authoritative anti-pattern instantiates when a tool's view of substrate state is treated as the answer to a coordination question; it also fails (b). The Pattern B bidirectional anti-pattern names the case where view changes are silently propagated back to substrate; it fails (a). The Pattern B view-as-authority anti-pattern names the case where a derived view is documented as derived but functions as authoritative in the system's actual behavior; it fails (b) by way of behavior rather than documentation. The stale-view-as-authority anti-pattern names the case where a view that has not been regenerated since the substrate changed is nevertheless treated as the answer; it fails (b) and (d) jointly.

Five architectural decisions follow as deployment requirements. First, derived views are documented as non-authoritative within the deployment's architecture description, and the documentation is itself substrate-readable so the architectural status of each view is inspectable as substrate content. Second, view-generation events record their provenance — substrate state at derivation, derivation procedure, procedure version, timestamp — as substrate-side records, so the derivation event is addressable on the same terms as substrate writes. Third, regenerability is exercised, not merely claimed: a view's regenerability is verified when the deployment can in fact rebuild the view from substrate state and observe equivalence within the determinism contract's allowed-non-determinism categories. Fourth, view-generation tooling is portable across the host environments the substrate's tool-agnosticism commitment supports; a view generatable only by a single vendor's tool has acquired a binding the architecture does not have. Fifth, the prohibition on bidirectional flow is enforced architecturally — any view-to-substrate writeback is redirected through a cell under an orchestration rule, converting the writeback into Pattern A input rather than allowing it to remain a Pattern B reverse channel.

6. Operational test

A composition instantiates Pattern B as architecture if and only if all of the following are true at all times during the substrate's existence:

1. The flow of view content runs in one direction only — from substrate state to the adjacent component as the view's content — and any propagation of view content back to substrate state runs through a cell under an orchestration rule.
2. The derived view is documented as non-authoritative for any coordination question the substrate is the source of truth for, and the deployment treats substrate-view disagreements as signals to refresh or rebuild the view rather than as questions to adjudicate.
3. The derivation event that produced the view's content is retraceable from substrate-side records, with provenance fields specifying source state, derivation procedure, procedure version, and timestamp.
4. The view is regenerable from substrate state under the derivation procedure of record, and regeneration is exercised — not merely claimed — at intervals appropriate to the deployment.
5. View-generation tooling does not bind the deployment to any host beyond what the substrate's tool-agnosticism commitment already requires.

The five are jointly necessary and individually load-bearing. The test admits three sharpening properties that focus its operational content where ambiguity is most likely.

(e.1) Pattern-B-unidirectional-derivation. A reader can verify that no path exists from view content to authoritative substrate state outside cell mediation. The verification is structural: the deployment's architecture either documents a route from view back to substrate (in which case the route is either through a cell — preserving unidirectionality of Pattern B by routing the view content through Pattern A as cell input — or it is a violation of the operationalization), or it documents no such route (in which case unidirectionality holds by construction).

(e.2) Pattern-B-non-authoritative-view. A reader can verify that the deployment treats the substrate, not the view, as the answer to coordination questions. The verification is behavioral: when the system answers "what was decided here," the answer comes from the substrate; when substrate and view disagree, the

substrate is what the system stands behind; when participants reach for the view first as a matter of habit, the architecture surfaces the substrate as the authoritative reference rather than allowing the habit to silently shift authority.

(e.3) Pattern-B-deterministic-regeneration. A reader can verify that the view is regenerable from substrate state under the derivation procedure of record, with the regeneration producing equivalent view content modulo the determinism contract's allowed-non-determinism categories. The verification is operational: the deployment can in fact rebuild the view, and the rebuilt view answers the same queries as the original within the equivalence class the determinism contract defines. Divergence beyond that class is treated either as evidence that the substrate state at regeneration differs from the substrate state of record (in which case the divergence is correct) or as evidence that the procedure is not in fact deterministic (in which case the operationalization fails at component (d)).

A composition that fails any of (1)–(5), or fails to satisfy any of (e.1)–(e.3) when sharpened, is not a Pattern B instance. It may be a useful composition; it may be a legitimate Pattern A or Pattern C composition mislabeled as Pattern B; or it may be one of the anti-patterns the operationalization preempts. The architectural test makes the distinction surfaceable at the moment a practical decision is made.

A one-sentence form: a composition is Pattern B if and only if substrate state is the unidirectional source from which a non-authoritative view is deterministically regenerable through a retraceable, vendor-portable derivation, with no route from view to substrate outside cell mediation.

7. Conclusion

Pattern B is the position the AI components most widely deployed in 2024–2026 systems can occupy legitimately. Search indexes, vector databases, retrieval-augmented generation surfaces over substrate-resident content, dashboards, materialized views, and cached projections are all Pattern B candidates; whether each is, in fact, a Pattern B instance depends on whether the four operational components hold for it, jointly, in the deployment's actual architecture rather than only in the deployment's documentation. The operationalization makes the distinction sharp, and the test makes it verifiable.

Naming Pattern B as standalone architectural specification — distinct from the general three-pattern framework, distinct from the specific composition pairs that touch on individual aspects of Pattern B, distinct from Pattern A and Pattern C — gives downstream implementations a single citable specification of what each Pattern B deployment must satisfy. The Pattern A operationalization in the prior note specifies cell-to-adjacent consultation; this note specifies substrate-derived view; the next will specify the separate-concern position; the closing tier note states the coherence relations across the three patterns as a system. Subsequent work that adopts, extends, composes, or argues against the CKS Pattern B position should use "Pattern B" in the operationalized sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Pattern B Operationalization: Substrate-Derived Views as Standalone Architectural Specification in the Coordination Knowledge Substrate Pattern.* May 7, 2026. ORCID: 0009-0004-8065-3235.